

HASHING

PROBLEM 7

QUESTION:

DS	Session	Hashing	Question Information	Level 1	Challenge 97
Problem Description Given an array A of N integers. Now, you have to output the sum of unique values of the maximum subarray sum of all the possible subarrays of the given array A. Note: Subarray means contiguous elements with at-least one element in it. Constraints $1 \leq N \leq 2000$ $0 \leq A_i \leq 10^9$ Input Format The first line of the input contains a single integer N, the total number of elements in array A. The next line of the input contains N space-separated integers representing the elements of the array. Output Format The only single line of the output should contain a single integral value representing the answer to the problem.					
Logical Test Cases Test Case 1 INPUT (STDIN) 4 5 -2 7 -3 EXPECTED OUTPUT 17 Test Case 2 INPUT (STDIN) 15 2 3 4 5 6 7 3 4 2 7 6 8 6 4 9 EXPECTED OUTPUT 2021					
Mandatory Test Cases Test Case 1 KEYWORD int NA[N];					
Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY 3 Test Case 2 TOKEN COUNT 555 Test Case 3 NLOC 73					

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

long long compute_kadane(int data[], int start, int end) {
    long long global_max = data[start];
    long long current_sub = data[start];
    for (int idx = start + 1; idx <= end; idx++) {
        current_sub = (current_sub + data[idx] > data[idx]) ? (current_sub + data[idx]) : data[idx];
        if (current_sub > global_max) global_max = current_sub;
    }
    return global_max;
}

int sort_longs(const void* a, const void* b) {
    long long diff = *(const long long*)a - *(const long long*)b;
    return (diff > 0) ? 1 : ((diff < 0) ? -1 : 0);
}

int main() {
    int length;
    if (scanf("%d", &length) != 1) return 0;

    int sequence[length];
    for (int i = 0; i < length; i++) {
        scanf("%d", &sequence[i]);
    }

    int total_windows = length * (length + 1) / 2;
    long long max_sum_list[total_windows];
    int write_ptr = 0;

    for (int i = 0; i < length; i++) {
        for (int j = i; j < length; j++) {
            max_sum_list[write_ptr++] = compute_kadane(sequence, i, j);
        }
    }

    qsort(max_sum_list, write_ptr, sizeof(long long), sort_longs);

    long long unique_aggregate = 0;
    for (int i = 0; i < write_ptr; i++) {
        if (i == 0 || max_sum_list[i] != max_sum_list[i - 1]) {
            unique_aggregate += max_sum_list[i];
        }
    }

    printf("%lld\n", unique_aggregate);
    return 0;
}
```

OUTPUT:

```
4
5 -2 7 -3
17
```

```
15
2 3 4 5 6 7 3 4 2 7 6 8 6 4 9
2021
```